

김민수



010-7714-9201



vibrio0102@gmail.com



linkedin



github

Core Skills

문제 정의와 기술 선정: 낯선 의료 도메인을 학습하고 연구자 인터뷰로 데이터 생산·공유의 병목을 발굴했습니다. 요구사항에 맞춰 기술별 책임을 나누고 사내 MLOps를 처음부터 설계·구축했습니다.

데이터 파이프라인과 재현 기반: Airflow · dbt · Iceberg · DataHub로 수집·변환·버전·계보를 연결하고, SeaweedFS REST Catalog와 PostgreSQL + pg_duckdb로 공통 조회 환경을 구축했습니다. manifest 동결·복원 및 MLflow 실험·모델 관리 기반을 구현했습니다.

온프레미스 인프라와 하드웨어 운용: 사내 베어메탈 Kubernetes 클러스터 구성을 단독 담당하고 12개 Pulumi 스택으로 관리합니다. 유휴 장비의 RAM·NVMe 재활용부터 네트워크 장비·NUMA·메모리·디스크 구성까지 직접 주도했습니다.

클라우드 데이터 엔지니어링과 협업: AWS EMR/PySpark 데이터 파이프라인과 벡터 검색을 구축하고 데이터 엔지니어링 팀을 리드했습니다. iOS·백엔드·프론트엔드 경험을 바탕으로 연구자와 플랫폼의 책임 범위를 구체화합니다.

Technical Skills

Languages: Python | Swift | JavaScript | Objective-C | Shell | SQL

Frameworks: PySpark | Flask | FastAPI | Kafka | Langchain | OpenAI | ReactJS | NextJS | Bun

laC · Platform: Pulumi | Kubernetes(kubeadm) | Cilium | SeaweedFS | Dex/OIDC | Docker

Data: Airflow | dbt(Cosmos) | DataHub

Lakehouse · OLAP: Apache Iceberg | DuckDB | ClickHouse

Database: MariaDB | PostgreSQL | PGVector | Firestore | Neo4J

ML: Ray/KubeRay | MLflow

Observability: Prometheus | Grafana | Loki | Vector

LLM: Langchain | LangGraph | OpenAI(Text Embedding) | RAG(Neo4J, PostgreSQL)

Data Analytics: CytoscapeJS | SigmaJS

iOS: ReactorKit | Async Await | Alamofire | Snapkit | Crashlytics | Kingfisher | SceneKit | SPM | React-Native

Cloud: AWS (EC2, Lambda, S3, IAM, EMR, Route53, RDS Aurora) | Firebase (Firestore, Storage) | GCP | Ubuntu | uv

Career Summary

데이터 엔지니어 (약 3년): 온프레미스 MLOps 신규 구축과 AWS 데이터 파이프라인 설계·운영 경험을 보유함. PySpark, Airflow, Vector DB 등으로 데이터 플랫폼 구축과 MLOps 파이프라인 설계를 주도함.

백엔드 엔지니어 (누적 약 2-3년): Python(Flask, FastAPI)으로 API 서버와 관리 시스템을 개발했고, WebSocket 실시간 통신과 Firebase 연동까지 직접 붙임.

프론트엔드 및 클라이언트 (약 4년): Next.js로 데이터 시각화 대시보드와 BI 툴을 만들었음. iOS 개발자로 커리어를 시작해 ReactorKit 기반 반응형 아키텍처를 설계하고 네이티브 앱을 개발함.

Experiences

HUINNO

Dec 2025 - Present

Data Engineer

서울

- 요구사항 발굴부터 MLOps 신규 구축까지 주도:** 연구자 인터뷰와 데이터·코드·실행 방식 분석으로 담당자 의존, 분산된 데이터, 버전·계보 부재를 요구사항으로 구체화했습니다. Airflow의 실행·재시도, dbt의 SQL 변환, Iceberg의 테이블 버전, DataHub의 계보 관리 역할을 정의하고 공통 플랫폼을 설계·구현했습니다.
- 온프레미스 베어메탈 Kubernetes 구축·운영:** 플랫폼 2인 팀에서 클러스터 구성을 단독 담당했습니다. 사내 장비로 10 노드 클러스터를 구축하고 네트워크·스토리지·인증·실행 환경을 12개 Pulumi 스택으로 코드화했습니다(2026.09 기준). 변경 이력과 스택 간 의존성을 관리해 인프라를 재구성할 수 있는 기반을 마련했습니다.

- **하드웨어 전반의 구성과 운용 주도:** 유후 PC·워크스테이션을 분해해 RAM·NVMe를 회수·재장착하고, 네트워크 장비 선정·교체까지 직접 수행했습니다. NUMA·듀얼채널 구성과 메모리 압박·swap을 점검하고 PostgreSQL 데이터 디스크를 분리하는 등 실제 워크로드에 맞춰 장비와 시스템을 함께 조정했습니다.
- **데이터 생산·전달의 암묵지를 공통 파이프라인으로 전환:** 각 노드에 흩어진 데이터와 버전 관리되지 않은 생성 코드, 일부 생산자에게 집중된 지식 때문에 담당자 부재 시 데이터 확보가 어려웠습니다. SSH 복사 중심 전달을 공용 저장소와 Airflow·dbt 기반 Bronze/Silver/Gold 파이프라인으로 전환하고 DataHub에 계보를 기록했습니다. 공통 프레임워크·연구용 데이터·MIMIC-IV·운영 ETL을 구현했습니다.
- **Iceberg 카탈로그와 DuckDB 조회 계층 설계:** SeaweedFS의 S3 호환 저장소와 Iceberg REST Catalog로 Silver/Gold 데이터·메타데이터를 관리하고 NVMe/HDD 티어를 구성했습니다. PostgreSQL + pg_duckdb 조회 환경을 마련하고, 차원 테이블 생성 시 카탈로그에서 최신 메타데이터 경로를 조회해 DuckDB에 전달하도록 구현했습니다. 파일명으로 버전을 추정하지 않고 카탈로그를 기준으로 데이터를 읽도록 했습니다.
- **데이터 버전과 실험·모델 재현 기반 구축:** 모델과 성능을 Notion으로 공유하고 데이터·생성 코드·실행 환경의 연결 기록이 없던 문제에 대응했습니다. 데이터 버전·코드 커밋·환경 정보 등을 담은 manifest 동결·복원 도구와 검증 절차를 구현하고, MLflow 트래킹·모델 레지스트리 및 SeaweedFS 아티팩트 저장소를 구축했습니다. 연구 코드 연동과 데이터부터 모델까지의 전체 재현 검증은 진행 중입니다.
- **운영 신뢰성과 연구자 역할 분담:** 누락 원본 데이터 재수집·무결성 검증과 환경별 서비스·버킷 분리를 적용했습니다. 연구자는 도메인 ETL을, 플랫폼은 실행·계보·검증·재시도를 담당하도록 경계를 정의하고 후속 인터뷰로 도입 계획을 조율했습니다. Dex/OIDC와 Kubernetes RBAC으로 접근 제어를 연결하고 KubeRay 공통 학습 환경을 구축해 연구 코드 이관을 지원하고 있습니다.

Cellkey

Manager, Data Engineer

Jul 2025 - Dec 2025

서초구, 서울

- **Apache Airflow 3.x**로 데이터 정제 파이프라인을 설계하고 로컬 환경에 배포함.
- **AWS Lambda·Batch·Step Functions**를 유지보수하고 최적화해 확장 가능한 데이터 워크플로우를 구축함.

Netmelon

Back-End Engineer

Mar 2025 - Jun 2025

Freelance

- Android·iOS 인앱 결제 API 구현(구독·구독 갱신·구독 취소)
- Flask 웹 서버를 Docker 이미지로 말아 Google Cloud Run에 배포
- Firebase 리소스의 소유권을 확인하는 기능 구현
- LLM 기반 API 라우트·서비스·함수의 단위 테스트 작성
- Firebase Firestore·Cloud Storage 운용
- 레거시 코드 리팩토링

(주) Eazel

Team Lead, AI Labs

Jun 2022 - Sep 2024

중구, 서울

- 팀 리더로 미술 시장 ETL 파이프라인을 총괄 — 아키텍처 설계부터 배포·모니터링·성능 최적화까지 주도
- Flask/NextJS 기반 대시보드 구축
- 옥션 데이터 스크래퍼 개발 및 운영
- Airflow로 데이터 수집·정제·적재를 주기 실행하는 파이프라인 구현
- AWS EMR에서 PySpark로 데이터 정제·적재 파이프라인 구현
- OpenAI·Facebook 임베딩 모델로 데이터 중복 제거 시스템을 개발하고, 벡터 DB를 적용해 정확도와 속도를 개선
- AWS 서비스 관리(EC2, IAM, S3, Route53, RDS, EMR)
- Github Actions로 CI/CD 구축
- LangChain·Streamlit으로 챗봇 프로토타입 구현
- 챗봇용 Neo4j Cypher 쿼리 도구와 PostgreSQL 쿼리 도구 구현 주도
- LangChain과 정보 추출 프롬프트로 데이터 스크래핑 구현

(주) Eazel

MLOps Engineer, MLOps

Oct 2021 - Jun 2022

중구, 서울

- HuggingFace NLP 등으로 데이터 분석 → 모델링 → 검증 전 과정을 직접 수행
- 인스타그램 데이터 스크래퍼 개발 및 운영
- Flask/NextJS/CytoscapeJS로 소셜 관계망 분석 대시보드를 개발해 분석 결과를 제공 — Series A 투자 유치에 기여
- DVC로 ML 라이프사이클 구현
- PostgreSQL로 데이터 관리

(주) Eazel

iOS Developer, iOS Team

Jan 2019 - Oct 2021

강남, 서울

- iOS 앱 성능과 확장성을 고려해 API 클라이언트와 연동 백엔드 로직 설계 주도
- Github Actions로 CI/CD 도입
- ReactorKit 기반 반응형 프로그래밍을 도입하고 아키텍처 설계를 주도해 앱 반응성과 유지보수성을 높임

(주) Finkle

Jan 2018 - Nov 2018

Software Engineer, DevOps

영등포, 서울

- 하이브리드 iOS, Android 앱 유지보수
- NodeJS 기반 데이터 스크래퍼 개발

Projects

(주) Eazel

Jan 2019 - Sep 2024

DataEngineer, AI Labs

Jun 2022 - Sep 2024

중구, 서울

Jan 2023 - Oct 2023

미술 시장 분석 서비스를 위한 데이터 파이프라인 개발

- 개발 리소스를 아끼려고 AWS Aurora RDS 한 대로 데이터 파이프라인 구축
- AWS EMR에서 PySpark로 데이터 가공 · 적재
- 분산 처리를 위한 파티셔닝 및 데이터 최적화
- 데이터 플랫폼(데이터 분석/MLOps) 총괄 설계 및 구축으로 Series B 투자 유치에 핵심 기여
- 대용량 처리가 가능한 데이터 파이프라인을 주도해 구축
- [Project Detail >>](#)

미술시장 데이터 스크래퍼 개발

Jan 2023 - Feb 2023

- requests와 BeautifulSoup4로 데이터 스크래퍼 개발
- 멀티스레딩으로 리소스를 빠르게 업로드하는 스크래퍼 개발
- LangChain 정보 추출 프롬프트로 정형 데이터 스크래핑 구현
- Swift로 작성하여 iOS 환경에서 동작하는 소셜 관계 수집 스크래퍼 개발
- boto3로 스크래퍼 인스턴스 50대를 동시에 관리
- WebSocket으로 관리 서버와 연결해 스크래퍼 인스턴스 제어
- 동적(Selenium, PlayWright), 정적(Public/Private API) 데이터 스크래퍼 개발 및 운용
- [Project Detail >>](#)

데이터 스크래퍼 중앙 관리 서버 개발

Jan 2023 - Feb 2023

- Flask로 스크래퍼를 중앙에서 관리하는 API 엔드포인트 개발
- WebSocket으로 스크래퍼와 연결해 수집 대상을 내려보내는 구조 구현
- SHA-256 해시로 로그인 자격 증명 전달
- Flask Blueprint로 엔드포인트 모듈화
- Nginx로 리버스 프록시 구성
- Route53과 Certbot으로 HTTPS 인증서 관리
- 스크래퍼를 한곳에서 관리하는 서버 개발 · 운영
- [Project Detail >>](#)

BI 툴 개발

Nov 2023 - Nov 2023

- Flask와 NextJS로 BI 툴 서버 개발
- 소셜 관계 데이터를 전달해주는 API 개발
- Cytoscape와 SigmaJS로 소셜 관계를 방향 그래프로 보여주는 BI 페이지 개발
- 데이터 라벨링을 위한 API 개발
- NextJS와 Carbon Design System으로 컴포넌트 개발
- Route53과 Certbot으로 HTTPS 인증서 관리
- Docker Compose로 EC2 인스턴스(Ubuntu 22.04)에 배포
- BI 툴을 화면부터 인프라까지(Nginx · NextJS · Flask) 직접 개발 · 운영
- [Project Detail >>](#)

Airflow 서비스 배포 & DAG 작성

Jan 2023 - Feb 2024

- Airflow PythonOperator로 관리 서버와 통신하는 스크래핑 스케줄링 DAG 작성

- boto3로 AWS EMR을 띄워 데이터 정제 · 적재를 돌리는 DAG 작성
- Sequential Executor로 DAG 운영
- Route53과 Certbot으로 HTTPS 인증서 관리
- Docker Compose로 EC2 인스턴스(Ubuntu 22.04)에 서비스 배포
- Airflow DAG로 스크래핑을 주기 실행하고 PySpark로 데이터 정제
- [Project Detail >>](#)

데이터 중복 제거 기능 개발

Nov 2023 - Feb 2024

- ML 리서치 엔지니어와 함께 Faiss-CPU로 IVFFlat 인덱스를 만들고, 임베딩 벡터 거리를 계산해 중복 제거 시스템 개발
- Facebook SSCD 모델로 이미지 임베딩 벡터 추출
- OpenAI 텍스트 임베딩 모델 Ada로 텍스트 임베딩 벡터 추출
- PostgreSQL pgvector 0.5.0 확장으로 인덱스 생성 오버헤드를 50% 줄임
- pgvector의 IVFFlat · HNSW 인덱스 생성
- 기존 numpy.array를 pgvector::vector로 전환
- 신뢰도 높은 데이터를 위한 Embedding 기반 데이터 정제
- [Project Detail >>](#)

효율적인 데이터 관리를 위해 데이터 웨어하우스 스키마 설계

Feb 2024 - Sep 2024

- 여러 소스에서 들어오며 생기는 중복 데이터를 관리하려고 SSoT 테이블을 포함한 데이터 웨어하우스를 DBDiagram으로 설계
- RDS 하나가 데이터 소스와 웨어하우스를 겸하며 커진 비용을 줄이려고, S3를 데이터 레이크로 쓰는 스키마로 재설계
- DataSource · DataWarehouse · DataMart · SSoT 스키마와 테이블 약 50개 설계
- 비용 절감 및 신뢰도 높은 데이터 관리를 위한 데이터 파이프라인 구조 재설계 주도
- [Project Detail >>](#)

AWS Service 관리

- IAM Role 기반 유저 별 서비스 접근 제어
- 비용을 줄이려고 EC2 인스턴스 타입을 x86에서 arm64로 전환
- 읽기 · 쓰기 인스턴스를 분리해 Aurora RDS PostgreSQL 운영
- 매니지드 서비스와 EC2 직접 운영의 비용을 따져 골라 씀
- 리소스별로 필요한 만큼만 권한 부여

MLOps Engineer, MLOps

Oct 2021 - Jun 2022

중구, 서울

미술시장 분석 PoC

Apr 2022 - Jun 2022

- HuggingFace NLP의 개체명 인식으로 아티스트 CV 데이터를 정량 분석하고 정제
- Word Histogram으로 기사 분석
- 감성 분석으로 기사 분석
- NLP로 텍스트 분석

인스타그램 관계 데이터 분석

Feb 2022 - Apr 2022

- Node2Vec · K-means · PCA로 관계 분석
- NextJS · Flask · CytoscapeJS로 관계를 방향 그래프로 보여주는 시각화 페이지 개발
- 분석 자료를 토대로 Series A 투자 유치
- 모델 추론에 넣을 형태로 데이터를 정제하고 피처를 추출해 추론 결과를 분석
- [Project Detail >>](#)

인스타그램 관계 정보 스크래퍼 개발

Oct 2021 - Jun 2022

- 데이터를 끊임 없이 모으기 위해 여러 방법을 구상하고 구현
- Selenium으로 동적 페이지 스크래퍼 개발
- requests로 Public API를 호출하는 스크래퍼 개발
- Instagram private API를 다루는 오픈소스 Instagrapi로 스크래퍼 개발
- Python으로 된 Instagrapi를 Swift로 옮겨 모바일에서 도는 스크래퍼 개발
- 소셜 관계 정보를 모으다 오픈소스에 기여: [Instagrapi](#)

- 막힌 문제를 될 때까지 다른 방법으로 두드림
- [Project Detail >>](#)

미술 아티스트 정보 스크래퍼 개발

Oct 2021 - Nov 2021

- requests로 정적 페이지 스크래퍼 개발
- boto3와 멀티스레딩으로 EC2 인스턴스를 여러 대 띄워 고속 스크래핑 환경 구성
- 아티스트 정보 60만 건 · CV 정보 300만 건 수집
- 이후 분석의 바탕이 된 데이터 확보

PostgreSQL 기반 수집 데이터 관리

- 스키마를 나눠 원본 데이터와 정제 데이터를 구분해 보관
- Python과 psycopg2로 데이터 정제
- 초기 형태의 데이터 파이프라인 구축
- [Project Detail >>](#)

iOS Developer, iOS Team

Jan 2019 - Oct 2021

강남, 서울

iOS 애플리케이션 개발 및 배포

- Fastlane과 Github Actions로 CI/CD 구축
- ARKit으로 작품을 벽에 걸어보는 Artwork Hanger 기능 구현
- Push Notification 기능 구현
- Alamofire로 백엔드 API를 호출하는 API 레이어 구축
- Unity Bridge로 Google Cardboard VR 기능 연동
- VR용 에셋을 AWS S3에서 내려받는 기능 구현
- 여러 건을 동시에 받을 수 있는 다운로드 서비스 구현
- 반응형 프로그래밍을 위해 ReactorKit을 도입하고 Model · View · Reactor 아키텍처를 구현
- 단위 테스트 환경 구축 및 점진적 커버리지 확대
- SnapKit으로 오토레이아웃 구현
- Kingfisher로 이미지 뷰 개발
- Pinterest 스타일 UICollectionView 개발
- 소규모 iOS 인하우스 앱은 혼자 만들 수 있음

(주) Finkle

Jan 2018 - Nov 2018

Software Engineer, DevOps

영등포, 서울

iOS Hybrid Application 유지보수

Jan 2018 - Nov 2018

- VueJS 웹과 Alert로 통신하는 UIWebView · WKWebView 기반 iOS 앱 유지보수
- Objective-C -> Swift 마이그레이션
- Alert으로 전달되는 ActionCode에 해당하는 기능 개발

Android Hybrid Application 유지보수

Jan 2018 - Nov 2018

- VueJS 웹과 Alert로 통신하는 Chrome WebView 기반 Android 앱 유지보수
- Java로 유지보수
- Alert으로 전달되는 ActionCode에 해당하는 기능 개발

NodeJS 기반 데이터 스크래퍼 개발

Jan 2018 - Nov 2018

- 기존 암호화 모듈을 재사용해 로그인 기능 구현
- 페이지네이션을 따라가며 스크래핑하는 기능 구현

Education

Coursera <u>Convolutional Neural Networks</u>	Feb 2023
Coursera <u>Structuring Machine Learning Projects</u>	Nov 2022
Coursera <u>Improving Deep Neural Networks: Hyperparameter Tuning, Regularization and Optimization</u>	Oct 2022
Coursera <u>Neural Networks and Deep Learning</u>	Sep 2022
정보처리기사 한국산업인력공단: 17203100440V	Nov 2017
정보처리운용기능사 한국산업인력공단: 09403054754L	Jul 2009
언어 • 영어: OPIc AL • 일본어: 일상 회화 가능 • 한국어: 모국어	
한국공학대학교 학사, 컴퓨터공학 (전체 GPA 3.4 · 전공 GPA 3.73 / 4.5) • Relevant Coursework: Data Structures, Algorithms (C), Operating System, Computer Architecture, Cloud Computing • <u>[프로보노] 사회적 약자를 위한 증강현실을 이용한 가구배치 시스템</u>	Mar 2010 - Feb 2018 시흥, 경기도

Others

구룡마을 연탄봉사	Jan 2018
Let us go iOS 해커톤 참가 OpenAI API 응답을 정해진 레시피 형식으로 받아오는 레이어 개발	Jun 2024
대한민국 육군 하사 만기제대 (전문하사) Visual Basic 매크로로 xlsx 임금 계산기 작성	Jun 2011 - Sep 2013

요구사항 발굴부터 MLOps 신규 구축까지 주도

- **요구사항 발굴부터 MLOps 신규 구축까지 주도:** 연구자 인터뷰와 데이터·코드·실행 방식 분석으로 담당자 의존, 분산된 데이터, 버전·계보 부재를 요구사항으로 구체화했습니다. Airflow의 실행·재시도, dbt의 SQL 변환, Iceberg의 테이블 버전, DataHub의 계보 관리 역할을 정의하고 공통 플랫폼을 설계·구현했습니다.
- **데이터 생산·전달의 암묵지를 공통 파이프라인으로 전환:** 각 노드에 흩어진 데이터와 버전 관리되지 않은 생성 코드, 일부 생산자에게 집중된 지식 때문에 담당자 부재 시 데이터 확보가 어려웠습니다. SSH 복사 중심 전달을 공용 저장소와 Airflow·dbt 기반 Bronze/Silver/Gold 파이프라인으로 전환하고 DataHub에 계보를 기록했습니다. 공통 프레임워크·연구용 데이터·MIMIC-IV·운영 ETL을 구현했습니다.

온프레미스 베어메탈 Kubernetes 구축·운영

- **온프레미스 베어메탈 Kubernetes 구축·운영:** 플랫폼 2인 팀에서 클러스터 구성을 단독 담당했습니다. 사내 장비로 10 노드 클러스터를 구축하고 네트워크·스토리지·인증·실행 환경을 12개 Pulumi 스택으로 코드화했습니다(2026.09 기준). 변경 이력과 스택 간 의존성을 관리해 인프라를 재구성할 수 있는 기반을 마련했습니다.
- **하드웨어 전반의 구성과 운용 주도:** 유후 PC·워크스테이션을 분해해 RAM·NVMe를 회수·재장착하고, 네트워크 장비 선정·교체까지 직접 수행했습니다. NUMA·듀얼채널 구성과 메모리 압박·swap을 점검하고 PostgreSQL 데이터 디스크를 분리하는 등 실제 워크로드에 맞춰 장비와 시스템을 함께 조정했습니다.

Iceberg 카탈로그와 DuckDB 조회 계층 설계

- **Iceberg 카탈로그와 DuckDB 조회 계층 설계:** SeaweedFS의 S3 호환 저장소와 Iceberg REST Catalog로 Silver/Gold 데이터·메타데이터를 관리하고 NVMe/HDD 티어를 구성했습니다. PostgreSQL + pg_duckdb 조회 환경을 마련하고, 차원 테이블 생성 시 카탈로그에서 최신 메타데이터 경로를 조회해 DuckDB에 전달하도록 구현했습니다. 파일명으로 버전을 추정하지 않고 카탈로그를 기준으로 데이터를 읽도록 했습니다.

데이터 버전과 실험·모델 재현 기반 구축

- **데이터 버전과 실험·모델 재현 기반 구축:** 모델과 성능을 Notion으로 공유하고 데이터·생성 코드·실행 환경의 연결 기록이 없던 문제에 대응했습니다. 데이터 버전·코드 커밋·환경 정보 등을 담은 manifest 동결·복원 도구와 검증 절차를 구현하고, MLflow 트래킹·모델 레지스트리 및 SeaweedFS 아티팩트 저장소를 구축했습니다. 연구 코드 연동과 데이터부터 모델까지의 전체 재현 검증은 진행 중입니다.
- **운영 신뢰성과 연구자 역할 분담:** 누락 원본 데이터 재수집·무결성 검증과 환경별 서비스·버킷 분리를 적용했습니다. 연구자는 도메인 ETL을, 플랫폼은 실행·계보·검증·재시도를 담당하도록 경계를 정의하고 후속 인터뷰로 도입 계획을 조율했습니다. Dex/OIDC와 Kubernetes RBAC으로 접근 제어를 연결하고 KubeRay 공통 학습 환경을 구축해 연구 코드 이관을 지원하고 있습니다.

(주) Eazel

Jan 2019 - Sep 2024

Data Engineer, AllLabs

Jun 2022 - Sep 2024
중구, 서울**미술품 투자 분석을 위한 통합 데이터 플랫폼 구축**

- **핵심 문제 및 목표:** 작가, 작품, 경매, 소셜 네트워크, 기관, 아티스트 CV 등 미술 시장의 모든 데이터가 여러 소스에 파편화되어 있었음. 이 데이터들을 포괄하는 신뢰도 높은 통합 데이터 파이프라인을 구축하고, 이를 AI 및 통계 기반의 데이터 분석 플랫폼으로 활용하여 Series B 투자 유치에 기여하는 것을 최종 목표로 함.
- **나의 역할:** 데이터 엔지니어링 팀 리드로서, 데이터 파이프라인의 아키텍처 설계부터 구현, 운영, LLM 에이전트 프로토타입 개발까지 프로젝트 전반을 주도함.
- **문제 해결 과정:**
 - * **트레이드오프 분석 (Pandas vs. PySpark):** 프로젝트의 최종 목표는 모든 데이터를 수집하는 것이었기에, 미래에 데이터 규모가 수억 건 이상으로 커질 가능성이 높다고 가정하고 아키텍처를 설계함. 현재의 빠른 개발보다 미래의 확장성(Scalability)과 신뢰성(Reliability)을 확보하기 위해, 초기 오버헤드를 감수하고 선제적으로 PySpark 도입을 결정함.
 - * **구체적인 구현 내용 (How):** AWS EMR 클러스터가 생성될 때, 각각의 데이터 처리 단계를 정의한 Python 파일들을 EMR Step으로 순차적으로 포함하여 실행하는 방식으로 데이터 정제 및 적재 파이프라인을 구현함. 이 파이프라인 내에서 Facebook의 SSCE 및 OpenAI Embedding 모델을 활용하고, PGVector의 HNSW 인덱스를 사용하여 중복 데이터를 필터링하는 시스템을 구축함.

- **결과 및 임팩트:**

- * **비즈니스 임팩트:** 구축된 고품질의 통합 데이터 플랫폼은 회사의 핵심 자산이 되었고, **Series B 투자 유치**를 성공시키는 데 **결정적인 역할**을 함.
- * **기술적 성과:** PySpark를 선제적으로 도입하여, 데이터가 급증했을 때 별도의 마이그레이션 없이 안정적으로 파이프라인을 운영할 수 있는 확장 가능한 아키텍처의 기반을 마련함.

[go back](#)

대규모 데이터 스크래퍼 중앙 관리 및 모니터링 서버 개발

- **핵심 문제 및 목표:** 미술 시장 데이터 수집을 위해 **동시에 50개 이상의 스크래퍼 인스턴스를 운영**하게 되면서, 각 인스턴스의 상태를 개별적으로 추적하고 관리하는 리소스 소모가 컸음. 여러 스크래퍼를 중앙에서 제어하고 실시간으로 모니터링하여 **데이터 수집 파이프라인의 안정성과 운영 효율성을 극대화**하는 것을 목표로 함.
- **나의 역할:** 서버의 전체 아키텍처 설계부터 API 엔드포인트 개발, WebSocket 통신, Nginx/HTTPS 설정 및 배포까지 **End-to-End 개발**을 모두 담당함.
- **문제 해결 과정:**
 - * **기술적 고민 및 선택 (Why):** 실시간 상태 모니터링을 위해, 클라이언트가 주기적으로 서버 상태를 요청하는 HTTP Polling 방식보다 서버와 클라이언트 간의 양방향 통신 채널을 유지하는 **WebSocket 방식** 이 서버 부하가 적고 상태 변화를 즉각적으로 전파할 수 있어 더 효율적이라고 판단함.
 - * **트레이드오프 분석 (Trade-off):** 경량 웹 프레임워크인 'Flask'와 'FastAPI'를 두고 고민함. 당시 프로젝트의 핵심은 안정적인 실시간 통신 구현이었기에, 관련 라이브러리 생태계가 성숙하고 제가 더 익숙했던 **'Flask'를 선택** 하여 개발 속도와 안정성을 우선으로 확보함.
 - * **구체적인 구현 내용 (How):** 'Flask'를 기반으로 스크래퍼 제어 API를 개발하고 **'Flask-Blueprint'로 코드를 모듈화**함. 'WebSocket'을 통해 각 스크래퍼와 연결을 유지하며 상태 및 메트릭을 실시간으로 교환하도록 구현함. 보안을 위해 로그인 정보는 **'SHA-256'으로 암호화** 하고, **'Nginx'를 리버스 프록시**로 사용하여 **'Certbot'으로 발급받은 HTTPS 인증서**를 적용함.
- **결과 및 임팩트:**
 - * **운영 효율성 극대화:** 수동으로 관리하던 50개 이상의 스크래퍼 인스턴스를 단일 API를 통해 중앙에서 관리할 수 있게 되어, 운영 리소스가 획기적으로 감소함.
 - * **안정성 및 신뢰성 향상:** 장애 발생 시 특정 인스턴스를 즉각적으로 파악하고 원격으로 재시작할 수 있게 되어, 데이터 수집 파이프라인 전체의 안정성과 데이터 누락 방지 신뢰성이 크게 향상됨.

[go back](#)

소셜 관계망 분석 및 데이터 정제를 위한 Full-Stack BI 툴 개발

- **핵심 문제 및 목표:** ML 모델이 분석한 소셜 관계망 데이터는 복잡한 그래프 형태로, 비개발자인 미술 전문가들이 직관적으로 이해하고 분석하기 어려웠음. 전문가가 직접 데이터를 검토하고 수정(라벨링)하여 데이터 품질을 개선하는 효율적인 도구가 필요했음.
- **나의 역할:** 데이터 구조 설계, API 개발(Flask), 프론트엔드(Next.js) 및 시각화(Cytoscape/ SigmaJS) 구현, 그리고 최종 배포(Docker, Nginx)까지 **전체 Full-Stack 개발**을 주도함.
- **문제 해결 과정:**
 - * **기술적 고민 및 선택 (Why):** 대규모 그래프 데이터를 브라우저에서 원활하게 렌더링하는 것이 핵심 과제였음. 초기에는 Cytoscape.js 를 사용했으나, 노드와 엣지가 늘어나자 성능 저하가 발생하여 WebGL 기반의 Sigma.js 로 마이그레이션하여 성능을 개선함.
 - * **트레이드오프 분석 (Trade-off):** UI 컴포넌트 개발 시, 범용 라이브러리 대신 IBM의 Carbon Design System 을 선택함. 국내 자료가 적어 초기 학습이 필요했지만, 데이터 시각화와 대시보드에 특화된 컴포넌트가 많아 장기적인 개발 생산성을 높일 수 있다고 판단함.
 - * **구체적인 구현 내용 (How):** Flask 로 소셜 관계 데이터와 데이터 라벨링을 위한 RESTful API를 개발함. Next.js와 Carbon Design System 으로 프론트엔드 환경 및 UI 컴포넌트를 구현함. 최종적으로 Docker Compose 를 이용해 Nginx, Next.js, Flask 서버를 컨테이너화하여 EC2에 배포함.
- **결과 및 임팩트:**
 - * **업무 효율성 향상:** 미술 전문가들이 DB 조회나 개발자 요청 없이, 직접 BI 툴을 통해 소셜 관계망 데이터를 탐색하고 분석할 수 있게 되어 업무 시간이 획기적으로 단축됨.
 - * **데이터 품질 개선:** 전문가가 시각화된 그래프를 보며 직접 데이터 오류를 수정하고 라벨링하는 기능을 제공하여, ML 모델의 학습 데이터 품질을 높이는 선순환 구조의 기반을 마련함.

[go back](#)

비용 효율적인 데이터 파이프라인 자동화를 위한 Airflow 구축 및 운영

- **핵심 문제 및 목표:** 데이터 수집과 정제 파이프라인이 수동으로 운영되어 비효율적이고 오류 발생 가능성이 높았음. 전체 데이터 파이프라인을 워크플로우로 정의하여 **실행, 모니터링, 재처리**를 자동화하고, **운영 비용을 최소화**하는 안정적인 스케줄링 시스템 구축을 목표로 함.
- **나의 역할:** Airflow 인프라 설계, EC2/Docker 기반 배포, 핵심 DAG(데이터 수집, EMR 처리) 작성 및 운영까지 워크플로우 자동화의 전 과정을 담당함.

- 문제 해결 과정:

- * **트레이드오프 분석 1 (인프라):** AWS Managed Airflow(MWAA)는 높은 비용 외에도, 당시 제가 선호하는 TaskFlow API를 사용할 수 없는 버전상의 제약이 있었음. 최신 버전을 자유롭게 사용하고 인프라 비용을 90% 이상 절감하기 위해, 초기 구축 리소스가 더 필요하더라도 EC2에 Docker Compose로 직접 배포하는 방식을 선택함.
- * **트레이드오프 분석 2 (Executor):** 병렬 처리를 위한 Celery Executor는 추가 인프라가 필요해 복잡성이 높다고 판단. 당시 Task의 순차적 안정성이 더 중요했기에, 가장 단순하고 안정적인 Sequential Executor를 선택하여 인프라 복잡도를 최소화함.
- * **구체적인 구현 내용 (How):** Docker Compose를 이용해 Airflow를 EC2에 배포. 스크래핑 DAG는 PythonOperator를 사용하여, 직접 로직을 실행하는 것이 아니라 외부의 스크래퍼 중앙 관리 서버의 API를 호출하여 작업을 스케줄링하는 방식으로 작성함. 데이터 정제 DAG는 boto3로 EMR 잡(Job)을 트리거하도록 구현함.

- 결과 및 임팩트:

- * **운영 효율성 극대화:** 매일 수동으로 실행하던 데이터 작업을 **완전 자동화**하여, 관련 운영에 소요되던 시간을 95% 이상 단축함.
- * **안정성 및 관측 가능성 향상:** Airflow UI를 통해 모든 파이프라인의 실행 상태와 로그를 중앙에서 모니터링할 수 있게 되어, 장애 발생 시 원인 파악 및 대응 시간이 크게 단축됨.

[go back](#)

임베딩 기반 이미지/텍스트 중복 제거 시스템 개발

- **핵심 문제 및 목표:** 초기 중복 제거 시스템은 DB에서 임베딩을 다운로드하고, 데이터 타입을 변환한 후, 외부에서 인덱스를 생성하는 **불필요한 오버헤드**가 발생. 이 비효율을 제거하고 DB 내에서 모든 로직을 처리하여 **파이프라인을 최적화**하고 데이터 품질을 높이는 것을 목표로 함.
- **나의 역할:** ML 엔지니어가 개발한 초기 시스템의 파이프라인 오버헤드를 분석하고, PGVector 기반의 새로운 아키텍처를 설계 및 도입하여 최적화하는 작업을 주도함.

- 문제 해결 과정:

- * **기술적 고민 (기존 방식의 문제점):** 초기 'Faiss-CPU' 기반 시스템은 PostgreSQL에서 데이터를 꺼내고, **호환되지 않는 데이터 타입을 변환한 후**, 로컬에서 인덱스를 생성해야 하는 비효율적인 구조였음.
- * **구체적인 구현 내용 (How):** 이 오버헤드를 해결하기 위해 모든 로직을 DB 내부에서 처리하도록 변경. 기존 'Numpy' 배열을 'pgvector::vector' 타입으로 마이그레이션하고, 'PGVector'의 'IVFFlat' 및 'HNSW' 인덱스를 생성하여 시스템을 재구축함.

- 결과 및 임팩트:

- * **운영 효율성 및 비용 절감:** 불필요한 데이터 전송 및 변환 과정을 완전히 제거하여, **전체 파이프라인의 오버헤드를 50% 이상 절감**하고 구조를 단순화함.
- * **데이터 품질 향상:** 최적화된 시스템을 통해 데이터의 정합성과 신뢰도를 획기적으로 향상시켜, 고품질의 미술 시장 데이터베이스 구축에 핵심적으로 기여함.

[go back](#)

비용 최적화 및 데이터 신뢰도 향상을 위한 데이터 웨어하우스 재설계

- **핵심 문제 및 목표:** 기존의 단일 RDS 중심 아키텍처는 **데이터 중복 및 불필요한 비용 문제**가 심각했음. 데이터 처리 단계별 저장소를 분리하고 SSoT(Single Source of Truth) 기반의 아키텍처를 도입하여, 데이터 신뢰도 향상과 운영 비용 절감을 목표로 함.
- **나의 역할:** 데이터 파이프라인의 문제점을 분석하고, 비용 절감 및 신뢰도 향상을 위한 **데이터 웨어하우스 구조 재설계**를 주도함.

- 문제 해결 과정:

- * **트레이드오프 분석 (RDS vs. S3 Data Lake):** 기존 RDS 구조는 초기 개발이 빠른 장점이 있으나, 장기적인 비용 및 확장성 문제가 명확했음. 초기 마이그레이션 비용을 감수하더라도, 저렴한 S3를 **데이터 레이크로 도입**하여 장기적인 스토리지 비용 절감 및 관리 유연성을 확보하는 방향으로 결정함.
- * **구체적인 구현 내용 (How):** DBDiagram 툴을 사용하여 DataSource, Data Warehouse, Data Mart 계층을 시각적으로 설계함. 데이터 중복을 통한 관리하기 위한 SSoT 테이블을 포함하여 **약 50개의 테이블 스키마를 설계**함.

- 결과 및 임팩트:

- * **비용 절감 기반 마련:** S3 데이터 레이크 도입을 통해 기존 아키텍처 대비 **예상 스토리지 비용을 획기적으로 절감**할 수 있는 기반을 마련함.
- * **데이터 신뢰도 향상:** SSoT 테이블 설계를 통해 데이터 중복을 체계적으로 관리하고, 데이터의 정합성과 신뢰도를 높일 수 있는 구조를 확립함.

[go back](#)

- **핵심 문제 및 목표:** 전문가의 정성적 판단에 의존하던 아티스트의 시장 영향력을 인스타그램 관계망 데이터로 객관화/정량화. 데이터 기반으로 성장 가능성을 증명하여, **Series A 투자 유치**를 성공시키는 것을 최종 목표로 함.
- **나의 역할:** MLOps 엔지니어로서, 데이터 수집부터 **DVC를 이용한 데이터/모델 버저닝**, 여러 모델의 입력값 가공, K-Means/PCA 분석 및 결과 시각화까지 PoC의 **재현 가능한 End-to-End 파이프라인 구축**을 주도함.
- **문제 해결 과정:**
 - * **기술적 고민 및 선택 (Why):** 당시 머신러닝 모델 자체에 대한 깊은 지식이 부족했기에, ML 리서치 엔지니어와 협업하여 각자의 전문 분야에 집중하는 방식을 선택. 동료는 **Node2Vec**, **GAT** 등 다양한 그래프 임베딩 모델링을, 저는 데이터 파이프라인과 후속 분석/시각화에 집중함.
 - * **구체적인 구현 내용 (How):** Git과 연동되는 **DVC**를 도입하여, 모델의 입력 데이터와 산출물을 Git 커밋 해시와 연동하여 버저닝하고 실험의 **재현성을 확보**함. 이후, 추출한 임베딩 벡터를 입력받아 **K-Means 클러스터링**과 **PCA**로 작가 그룹을 분석함. 최종적으로 이 분석 결과를 **Next.js** 및 **Cytoscape.js**를 활용하여 상호작용 가능한 네트워크 시각화 대시보드로 구현함.
- **결과 및 임팩트:**
 - * **비즈니스 임팩트:** 개발된 시각화 대시보드의 분석 자료를 통해 아티스트의 성장 가능성을 증명, **Series A 투자 유치**를 성공시키는 데 **결정적인 기여**를 함.
 - * **개인 및 기술적 성장:** 이 협업 경험을 통해 머신러닝 파이프라인 구축에 대한 깊은 실무 이해를 얻었으며, 데이터 엔지니어로서의 커리어로 본격적으로 전환하는 결정적인 계기가 됨.

[go back](#)

인스타그램 봇 탐지 우회를 위한 데이터 스크래퍼 개발 및 고도화

- **핵심 문제 및 목표:** 인스타그램의 강력한 **봇 탐지(Bot Detection)** 시스템으로 인해 데이터의 지속적인 수집이 불가능한 문제를 해결. 봇 탐지를 안정적으로 우회하여, 대규모 소셜 관계 데이터를 **중단 없이 수집**할 수 있는 견고한 스크래퍼 개발을 목표로 함.
- **나의 역할:** 봇 탐지 문제 해결을 위한 다양한 기술적 접근법(**Selenium**, **requests**, **Instagrapi** 등)을 리서치, 프로토타이핑하고, 최종 솔루션을 구현하는 역할을 담당함.
- **문제 해결 과정:**
 - * **트레이드오프 분석 (서버 vs. 모바일 환경):** 서버 환경에서 Python 기반의 스크래퍼를 사용하는 것은 결국 봇 탐지에 의해 차단됨을 확인함. '서버 환경' 자체가 탐지의 원인이라는 가설 하에, 초기 비용이 높더라도 **Python 라이브러리**를 **Swift**로 직접 **마이그레이션**하여 모바일 환경 요청을 흉내 내는 근본적인 해결책을 선택함.
 - * **구체적인 구현 내용 (How):** Python으로 작성된 '**Instagrapi**'의 **핵심 로직**을 **Swift 언어**로 직접 포팅하여, iOS 모바일 환경에서 동작하는 데이터 스크래퍼를 개발함. 개발 과정에서 발견한 라이브러리 버그를 수정하여 **오픈소스 프로젝트**에 직접 기여함.
- **결과 및 임팩트:**
 - * **핵심 문제 해결:** 기존 방법으로는 불가능했던 **대규모 소셜 관계 데이터를 안정적으로 수집**하는 데 성공하여, 'MLOps 분석 PoC' 프로젝트의 기반을 마련함.
 - * **문제 해결 역량 입증:** 프레임워크를 다른 언어로 포팅하는 창의적인 접근을 통해, 고질적인 외부 시스템의 제약사항을 해결함.

[go back](#)

PostgreSQL 스키마 분리를 이용한 초기 데이터 파이프라인 구축

- **핵심 문제 및 목표:** 수집한 원본(Raw) 데이터와 정제된(Purified) 데이터를 효율적으로 분리 및 관리할 필요가 있었음. 별도의 웨어하우스 구축이 어려운 상황에서, **단일 DB 내에서 데이터 무결성을 보존하고 안정적으로 데이터를 제공하는** 실용적인 파이프라인 구축을 목표로 함.
- **나의 역할:** 데이터 정제 로직을 개발하고, **PostgreSQL 내에서 스키마를 분리하여 데이터 계층을 나누는 아키텍처**를 직접 설계하고 구현함.
- **문제 해결 과정:**
 - * **트레이드오프 분석 (DB 분리 vs. 스키마 분리):** 물리적인 DB 분리는 비용과 관리 포인트가 2배로 늘어나는 단점이 있었음. 대신, 추가 비용 없이 논리적 분리가 가능한 **PostgreSQL의 스키마 분리 방식**을 선택하여 실용성과 비용 효율성을 확보함.
 - * **구체적인 구현 내용 (How):** PostgreSQL 내에 **raw_data**와 **purified_data** 스키마를 생성. **Python의 psycopg2 라이브러리**를 사용하여 데이터 정제 로직을 수행하고, 각 스키마의 테이블에 데이터를 저장하는 파이프라인을 구축함.
- **결과 및 임팩트:**
 - * **데이터 무결성 확보:** 원본 데이터는 변경 없이 보존하고, 분석가는 정제된 데이터만 안전하게 사용할 수 있는 환경을 구축하여 데이터 관리의 복잡성을 낮춤.
 - * **성장의 발판 마련:** 이 경험을 통해 데이터 웨어하우스의 필요성을 깨닫고, 이후 더 고도화된 데이터 아키텍처를 설계하는 역량의 기반을 다짐.

[go back](#)

iOS 애플리케이션 개발 및 배포

iOS Developer

- **핵심 문제 및 목표:** 미술 관련 콘텐츠를 네이티브 앱에서 보여주고, VR 감상 · 에셋 다운로드 · AR 기능까지 담은 iOS 앱을 만드는 것이 목표.
- **나의 역할:** API 클라이언트를 포함한 백엔드 레이어를 주도해 구축. 작품 · 작가 · 전시 상세 View와 ViewController를 구현했고, Unity Bridge로 Google Cardboard VR을, ARKit으로 Artwork Hanger를 붙였으며, VR 에셋을 내려받는 다운로드 기능도 구현.
- **문제 해결 과정:**
 - * **기술적 고민 및 선택 (Why):** VR 에셋 다운로드, Unity 연동 등 복잡한 기능 추가를 앞두고, 기존 MVC 구조의 복잡성 증가 및 유지보수성 저하를 예상. 상태(State)를 보다 명확하고 예측 가능하게 관리할 수 있는 반응형 아키텍처 도입을 목표로 함.
 - * **트레이드오프 분석 (Trade-off):** 직접 MVVM 패턴을 구현하는 방안을 먼저 고려함. 하지만 이는 상태와 사용자 입력을 처리하기 위한 프로토콜 정의, 데이터 바인딩 로직 구현 등 상당한 초기 개발 리소스가 필요하다고 판단. 대안으로, 반응형 프로그래밍 프레임워크인 **ReactorKit**을 검토함. ReactorKit은 **사용자 입력(Action) -> 비즈니스 로직 처리(Mutate) -> 상태 변경 적용(Reduce)**으로 이어지는 3단계의 명확한 데이터 흐름을 강제하여, 직접 구현 시 발생할 수 있는 복잡성을 원천적으로 차단하고 코드의 흐름을 매우 쉽게 이해할 수 있도록 만들어주는 점이 인상적이었음. 초기 학습 곡선을 감수하더라도, ReactorKit을 도입하는 것이 장기적인 유지보수성과 개발 효율성 측면에서 더 유리하다고 결론 내림.
 - * **구체적인 구현 내용 (How):** ReactorKit을 기반으로 앱의 아키텍처를 재설계. 모든 사용자 입력은 'Action'으로, 상태 변화는 'State'로 정의하여 데이터 흐름을 통일하고, AWS S3 클라이언트를 이용한 다운로드 서비스를 Reactor에 통합하여 구현.
- **결과 및 임팩트:**
 - * **개발 효율성 향상:** 프레임워크에 익숙해진 후, **Action-Mutate-Reduce** 구조에 맞춰 로직을 정의하는 과정이 매우 쉬워지고 명확해짐. 이로 인해 신규 기능 추가 시 개발 속도가 향상되고 코드의 일관성이 유지됨.
 - * **안정성 및 유지보수성 확보:** 명확한 구조 덕분에 상태 관리 관련 버그 발생률이 감소했으며, 새로운 팀원이 코드 구조를 파악하는 시간이 단축됨.